

# Evaluation of Bitwise Operations vs. Plain Old Data (POD) for Efficient Packet Header Serialization in C++

November 30, 2025

## Contents

|          |                                                                                                         |          |
|----------|---------------------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Evaluation of Bitwise Operations vs. Plain Old Data (POD) for Efficient Packet Header Serializa-</b> | <b>2</b> |
| 1.1      | Purpose and Scope . . . . .                                                                             | 2        |
| <b>2</b> | <b>Manual Serialization via Bitwise Operations</b>                                                      | <b>2</b> |
| 2.1      | Foundational Operators . . . . .                                                                        | 2        |
| 2.2      | Implementation Strategy for Buffer Manipulation . . . . .                                               | 3        |
| 2.3      | Case Study: MacHdr Serialization . . . . .                                                              | 3        |
| 2.4      | Analysis of the Bitwise Method . . . . .                                                                | 4        |
| <b>3</b> | <b>Serialization using Plain Old Data (POD) Structures with Bitfields</b>                               | <b>4</b> |
| 3.1      | The POD (Plain Old Data) Bitfield Declaration . . . . .                                                 | 4        |
| 3.2      | Implementation via <code>memcpy</code> . . . . .                                                        | 5        |
| 3.3      | Analysis of the POD Method . . . . .                                                                    | 5        |
| <b>4</b> | <b>Comparative Evaluation and Recommendations</b>                                                       | <b>5</b> |
| 4.1      | Recommendations . . . . .                                                                               | 5        |
| <b>5</b> | <b>C++ I/O Streams for Data Persistence</b>                                                             | <b>6</b> |
| 5.1      | Overview and Binary Mode Operation . . . . .                                                            | 6        |
| 5.2      | Reading and Writing Binary Buffers . . . . .                                                            | 6        |

# 1 Evaluation of Bitwise Operations vs. Plain Old Data (POD) for Efficient Packet Header Serialization in C++

## 1.1 Purpose and Scope

Efficient data serialization is a critical concern in **network engineering**, directly impacting the performance, latency, and resource utilization of our applications. The process of converting in-memory data structures into a byte stream suitable for network transmission must be both **fast** and **precise**. This memorandum establishes our engineering guidelines for selecting a serialization technique by analyzing and comparing two fundamental C++ approaches for network packet headers: manual bitwise manipulation and the use of **Plain Old Data (POD)** structures with bitfields.

The evaluation of these methods will be based on the following key criteria:

- **Performance:** The computational efficiency of the serialization and deserialization process.
- **Memory Efficiency:** The memory footprint of the serialized data and the structures used to manage it.
- **Implementation Complexity:** The level of effort, code verbosity, and potential for error during development and maintenance.

This document will begin with a detailed examination of manual serialization using bitwise operations, a technique that offers maximum control over data layout.

## 2 Manual Serialization via Bitwise Operations

Manual serialization using **bitwise operators** is a low-level, high-control approach that enables engineers to construct a data buffer one bit at a time. This method is strategically important when dealing with non-standard packet layouts, protocols requiring specific bit-level packing, or when ensuring absolute **cross-platform consistency** is paramount. It allows for the creation of a serialized representation that is completely independent of compiler-specific memory alignment or padding rules.

### 2.1 Foundational Operators

The foundation of this technique rests on C++'s native bitwise operators, which manipulate data at the individual bit level.

Table 1: C++ Bitwise Operators

| Operator     | Description & Example                                                                                                                                                                        |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| & (AND)      | Performs a bitwise AND operation. <code>uint_fast8_t a = 11; // 0000 1011, uint_fast8_t b = 5; // 0000 0101, a &amp; b results in 1 (0000 0001).</code>                                      |
| (OR)         | Performs a bitwise OR operation. <code>uint_fast8_t a = 11; // 0000 1011, uint_fast8_t b = 5; // 0000 0101, a   b results in 15 (0000 1111).</code>                                          |
| ^ (XOR)      | Performs a bitwise exclusive OR (XOR) operation. <code>uint_fast8_t a = 11;, uint_fast8_t b = 5;, a ^ b results in 14 (0000 1110).</code>                                                    |
| ~ (NOT)      | Performs a bitwise complement (unary), flipping each bit. <code>uint_fast8_t b = 5; // 0000 0101, ~b results in 250 (1111 1010).</code>                                                      |
| << (L-Shift) | Shifts bits to the left, discarding the leftmost bits and padding with zeros on the right. <code>unsigned int a = 11; // ...1011, a &lt;&lt; 2 results in 44 (...101100).</code>             |
| >> (R-Shift) | Shifts bits to the right, discarding the rightmost bits. For unsigned types, pads with zeros on the left. <code>unsigned int a = 11; // ...1011, a &gt;&gt; 2 results in 2 (...0010).</code> |

## 2.2 Implementation Strategy for Buffer Manipulation

To manually serialize data into a character buffer, one must be able to access and modify individual bits within that buffer. Assuming a linear sequence of bits starting at *offset* = 0, the core logic is as follows:

- To retrieve the bit value at a given *offset*:

$$(buffer[(int)[offset/8]] >> (7 - offset\%8)) \& 1U$$

Here,  $1U$  is used to ensure the literal 1 is treated as an unsigned integer, preventing unintended sign extension and ensuring a correct bitwise AND operation.

- To set a bit to 1 at a given *offset*:

$$buffer[(int)[offset/8]] |= 1U << (7 - offset\%8)$$

In both operations,  $[offset/8]$  calculates the index of the target byte (`char`) in the buffer, while  $(7 - offset\%8)$  calculates the bit's position within that byte, assuming a **most-significant-bit-first (MSB)** ordering where bit 0 is the leftmost bit.

## 2.3 Case Study: MacHdr Serialization

Consider a simple MAC header structure that needs to be serialized into a buffer using the formulas defined above.

```
struct MacHdr {
    uint_fast8_t sn = 27;
    uint_fast8_t src = 1;
    uint_fast8_t dest = 3;

    const size_t sn_size = 7;    // 7 bits
    const size_t addr_size = 6;  // 6 bits for src and dest

    bool serialize(char* buffer, size_t& offset) const;
};
```

The `serialize` function iterates through the bits of each field, extracts them, and places them into the correct position in the output buffer using the logic from Section 2.2.

```
bool MacHdr::serialize(char* buffer, size_t& offset) const {
    // Helper lambda to serialize a single value
    auto serialize_field = [&](uint_fast8_t value, size_t num_bits) {
        for (int i = 0; i < num_bits; ++i) {
            // Extract the i-th bit from left-to-right (MSB-first)
            bool bit_is_set = (value >> (num_bits - 1 - i)) & 1;

            if (bit_is_set) {
                // Set the corresponding bit in the buffer
                buffer[(int)floor(offset / 8)] |= 1U << (7 - offset % 8);
            }
            // A 'clear bit' operation would be needed if buffer isn't zero-initialized
            offset++;
        }
    };

    serialize_field(sn, sn_size);
    serialize_field(src, addr_size);
    serialize_field(dest, addr_size);

    return true;
}
```

This implementation correctly applies the MSB-first logic. For each bit of each field, it is first isolated and then, if set, the corresponding bit is flipped to 1 in the destination buffer at the correct *offset*.

## 2.4 Analysis of the Bitwise Method

- **Pros:**
  - **Complete Control:** This method provides fine-grained, bit-level control over the data layout, which is essential for conforming to complex or non-standard protocol specifications.
  - **Compiler Independence:** The resulting byte stream is not subject to compiler-specific padding, alignment, or endianness rules, ensuring maximum portability and consistency across different platforms and architectures.
- **Cons:**
  - **High Complexity:** The logic is verbose, requires careful manual calculation, and is highly susceptible to off-by-one errors and other subtle bugs.
  - **Error-Prone:** Manual bit manipulation is tedious and can be difficult to debug and maintain over time.
  - **Performance Overhead:** This method incurs performance overhead due to bit-by-bit looping, which prevents CPU optimizations like vectorization that are possible with bulk memory operations.

This inherent complexity motivates the search for a higher-level abstraction.

## 3 Serialization using Plain Old Data (POD) Structures with Bitfields

The **POD with bitfields** approach provides a higher-level abstraction, shifting the burden of memory layout from the engineer to the compiler. This declarative method allows an engineer to define the desired bit-level layout directly in the `struct` definition. For standard packet structures, this can dramatically simplify the serialization process.

### 3.1 The POD (Plain Old Data) Bitfield Declaration

Bitfields allow struct members to be defined with a specific number of bits using the syntax `type variable : number_of_bits;`. For this to be effective for serialization, the structure must only contain integral types.

The `MacHdr` from the previous example can be refactored using bitfields:

```
struct MacHdr {
    uint_fast8_t sn : 7; // 7 bits
    uint_fast8_t src : 6; // 6 bits
    uint_fast8_t dest : 6; // 6 bits
};
```

This declarative style is also powerful for defining standard, more complex headers. For instance, a simplified TCP header can be clearly defined, including reserved or unused bits designated by unnamed fields:

```
struct SimpleTcpHeader {
    int source_port : 16;
    int destination_port : 16;
    int sequence_number : 32;
    int ack_number : 32;
    char data_offset : 4;
    char : 3; // 3 unused bits
    bool ns : 1;
    bool crw : 1;
    bool ece : 1;
    bool urg : 1;
    bool ack : 1;
    bool psh : 1;
    bool rst : 1;
    bool syn : 1;
```

```

    bool fin           : 1;
    int  window_size   : 16;
    int  checksum       : 16;
    int  urgent_pointer : 16;
};

```

### 3.2 Implementation via `memcpy`

The primary advantage of the POD bitfield approach is that if the compiler arranges the struct in memory exactly as required by the network protocol, serialization and deserialization become trivial. The entire operation can be reduced to a single, highly efficient memory copy using `memcpy`.

```

#include <string.h>

// Assume buffer and hdr1/hdr2 instances are defined
MacHdr hdr1 = {5, 4, 3};
MacHdr hdr2;
char buffer[10];
size_t offset1 = 0;
size_t offset2 = 0;

// To serialize hdr1 into the buffer:
memcpy(buffer + offset1, &hdr1, sizeof(hdr1));
offset1 += sizeof(hdr1);

// To deserialize from the buffer into hdr2:
memcpy(&hdr2, buffer + offset2, sizeof(hdr2));

```

### 3.3 Analysis of the POD Method

- Pros:
  - **Simplicity & Readability:** The code is declarative, concise, and significantly easier to write and understand than manual bitwise operations.
  - **High Performance:** `memcpy` is typically a highly optimized intrinsic function that can copy memory far more efficiently than bit-by-bit loops.
  - **Memory Efficiency:** Packing data into bitfields can result in a smaller memory footprint for the struct instance itself compared to using full-sized integer types for each field.
- Cons:
  - **Compiler-Dependent Layout:** The primary drawback is reliance on compiler-specific behavior. The C++ standard does not guarantee the exact layout, padding, or ordering of bitfields, which can lead to portability issues across different compilers or platforms.
  - **Addressability Limitation:** It is not possible to take the address of a bitfield member (e.g., `&hdr.sn`), as it may not align to a byte boundary.

This method sets the stage for a direct comparison to determine the appropriate use case for each technique.

## 4 Comparative Evaluation and Recommendations

Synthesizing the analysis from the previous sections provides a clear framework for choosing the appropriate serialization technique. The decision hinges on a direct trade-off between control, portability, performance, and development effort.

### 4.1 Recommendations

Based on this evaluation, the following guidelines are established to direct technical decisions:

Table 2: Comparison of Serialization Methods

| Evaluation Criterion             | Bitwise Operations                                                                                                                                                            | POD with Bitfields                                                                                                                                     |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Performance</b>               | Potentially lower due to overhead from bit-by-bit looping and conditional logic.                                                                                              | Generally <b>high</b> , as serialization is a single, highly optimized <code>memcpy</code> operation.                                                  |
| <b>Memory Usage</b>              | The pre-serialization struct often uses standard integer types (e.g., <code>int</code> ), potentially leading to a larger in-memory footprint than the final serialized data. | The struct instance itself is <b>memory-compact</b> , as the compiler packs fields into the smallest possible space, reducing the in-memory footprint. |
| <b>Implementation Complexity</b> | <b>High.</b> The code is verbose, algorithmically complex, and prone to subtle, hard-to-debug errors.                                                                         | <b>Low.</b> The approach is declarative, concise, and leverages a standard library function for the heavy lifting.                                     |

- Teams **should default** to POD with bitfields for implementing standard, well-defined network protocols where the header layout is fixed and broadly supported (e.g., TCP/IP, Ethernet). This approach is ideal when development speed, code simplicity, and raw copy performance are the primary concerns. It is the preferred method for internal or platform-specific applications where compiler consistency is not a risk.
- Manual bitwise operations **are mandated** for protocols requiring absolute, bit-level control over the data layout to ensure compatibility across diverse systems, architectures, or compilers. This includes complex, non-standard, or proprietary protocols. It is the required choice for applications that must be highly portable and adhere strictly to a protocol specification that may not align with a specific compiler's bitfield layout.

## 5 Ancillary Topic: C++ I/O Streams for Data Persistence

Once data is serialized into a buffer, a common next step is to persist it to storage or retrieve it from a file. The C++ standard library provides the `<fstream>` header for robust file input/output operations. The raw byte buffers produced by either serialization method are prime candidates for file I/O.

### 5.1 Overview and Binary Mode Operation

The `<fstream>` library provides several classes for file handling, primarily `ifstream` (input file stream) and `ofstream` (output file stream). When dealing with raw serialized data, it is critical to open files in **binary mode**.

```
#include <fstream>
std::ofstream fout("packet_data.bin", std::ios::binary);
std::ifstream fin("packet_data.bin", std::ios::binary);
```

Using `std::ios::binary` prevents the operating system from performing any unintended text formatting or character translation (e.g., converting line endings), which would corrupt the raw byte stream.

### 5.2 Reading and Writing Binary Buffers

The `read()` and `write()` methods are used to move blocks of binary data between a `char*` buffer and the file stream.